

Classification linéaire

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique
- 6 Récapitulatif

Rappels :

$$h_w(x) = w^T x$$

$$w = (X^T X)^{-1} X^T y$$

- Le **modèle linéaire** : un produit scalaire sur l'entrée augmentée.
- Sa **forme close**, obtenue par maximum de vraisemblance sous bruit gaussien.
- Le **maximum a posteriori** ajoute un a priori sur w , c'est-à-dire une pénalité $\lambda \Omega(w)$.
- L'**entropie croisée** $-\sum_x p(x) \log q(x)$.

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique
- 6 Récapitulatif

De la régression à la classification

Jusqu'ici, la cible était un **nombre** et on lisait le score $w^T x$ directement.

Maintenant la cible est une **étiquette** : spam ou non, malin ou bénin, chiffre 0 à 9.

Le modèle ne change pas

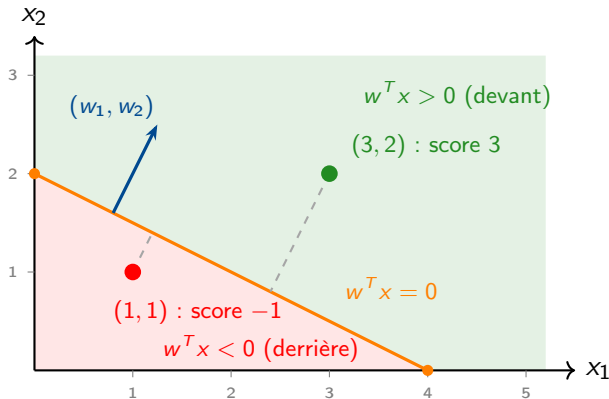
On garde exactement le même score linéaire. Seule change la façon de le *lire* : on n'en retient que le **signe**.

Un classifieur linéaire attribue la classe d'après le signe du score, et l'ensemble des entrées de score nul est la **frontière de décision** :

$$h_w(x) = \text{sign}(w^T x), \quad w^T x = 0 \text{ est la frontière}$$

- La frontière est un **hyperplan** : une droite avec deux caractéristiques, un plan avec trois.
- Le **signe** du score dit de quel côté de l'hyperplan l'entrée tombe.
- Sa **grandeur** dit à quelle distance de la frontière elle se trouve, donc avec quelle confiance on décide.

Hyperplan séparateur



Avec $w = (-4, 1, 2)$, la frontière est la droite $-4 + x_1 + 2x_2 = 0$, normale à (w_1, w_2) . La grandeur du score croît avec la distance à la frontière (pointillés).

Exemple

Avec $w = (-4, 1, 2)$, le biais en tête sur l'entrée augmentée $x = (1, x_1, x_2)$:

$$x = (3, 2) \Rightarrow w^T x = -4 + 3 + 4 = 3 \quad x = (1, 1) \Rightarrow w^T x = -4 + 1 + 2 = -1$$

- Score **positif** : le premier point tombe devant l'hyperplan, classe $+1$.
- Score **négatif** : le second tombe derrière, classe -1 .
- Le premier a le plus grand score en valeur absolue : il est plus loin de la frontière, donc classé avec plus de confiance.

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique
- 6 Récapitulatif

Notons les deux classes $y \in \{-1, +1\}$ et traitons-les comme des **cibles de régression**. On peut alors appliquer le cours précédent tel quel, forme close comprise :

$$w = (X^T X)^{-1} X^T y, \quad h_w(x) = \text{sign}(w^T x)$$

Moindres carrés en multiclasse

Pour $K > 2$ classes, chaque étiquette devient une ligne **one-hot** de $Y \in \mathbb{R}^{m \times K}$: un 1 dans la colonne de sa classe, des 0 ailleurs (la classe 2 sur trois s'écrit $(0, 1, 0)$). Les prédictions multiples de la régression se réutilisent :

$$W = (X^T X)^{-1} X^T Y, \quad \hat{y} = \arg \max_k (W^T x)_k$$

- Une colonne de W par classe, donc un hyperplan par classe.
- On prédit la classe au **score le plus élevé**, comme on l'a fait pour les K sorties d'une régression.

Exemple (1/2)

Trois classes, six exemples, deux caractéristiques :

$$X = \begin{array}{c} \begin{array}{ccc} x_0 & x_1 & x_2 \end{array} \\ \left[\begin{array}{ccc} 1 & -2 & -1 \\ 1 & -2 & 1 \\ 1 & 1 & -2 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \\ 1 & 1 & 2 \end{array} \right] \end{array}$$

$$Y = \begin{array}{c} \begin{array}{ccc} c_1 & c_2 & c_3 \end{array} \\ \left[\begin{array}{ccc} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{array} \right] \end{array}$$

Exemple (2/2)

$$X^T X = \begin{bmatrix} 6 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 12 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 2 & 2 & 2 \\ -4 & 2 & 2 \\ 0 & -3 & 3 \end{bmatrix}$$

$$W = \begin{bmatrix} c_1 & c_2 & c_3 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ -\frac{1}{3} & \frac{1}{6} & \frac{1}{6} \\ 0 & -\frac{1}{4} & \frac{1}{4} \end{bmatrix}$$

Prédiction

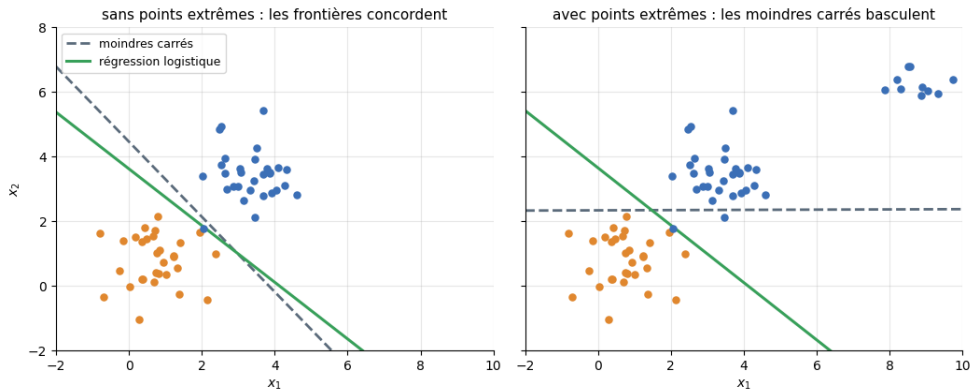
$x = (2, -2)$ donne $W^T x = (-\frac{1}{3}, \frac{7}{6}, \frac{1}{6})$, donc $\hat{y} = 2$.

Inconvénient 1 : les points bien classés tirent la frontière

La perte quadratique pénalise les grands scores.

- Un point très bien classé a un score $w^T x \gg 1$, donc un gros résidu par rapport à sa cible $+1$.
- Les moindres carrés le comptent comme une **erreur** et déplacent la frontière pour la réduire.
- Autrement dit : *les points qui font le moins de doute tirent sur la frontière.*

Moindres carrés contre régression logistique



Sans points extrêmes, moindres carrés et régression logistique concordent. Ajouter des points lointains et pourtant bien classés fait basculer la frontière des moindres carrés dans l'erreur, tandis que la régression logistique bouge à peine.

Inconvénient 2 : la forme close suppose une distribution

Hypothèse de distribution

La forme close n'est pas gratuite. Elle vient d'une **hypothèse de distribution**, et quand cette hypothèse est fausse le modèle se dégrade de façon visible.

Les algo 2 et 3 ne supposent rien sur la distribution des classes.

Contrepartie

Aucune des deux n'a de forme close. Il faut les ajuster **itérativement**, d'où la descente de gradient.

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient**
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique
- 6 Récapitulatif

Solution itérative

Si on ne peut pas résoudre analytiquement, on résout **pas à pas** : partir d'un w quelconque et l'améliorer jusqu'à ce que la perte cesse de baisser.

La descente de gradient est le **pilier** du machine learning moderne et de tout le deep learning.

Montagne dans le brouillard

Imaginez que vous êtes sur une montagne, de nuit, dans le brouillard.

- Vous voulez atteindre la vallée la plus basse (le coût minimum).
- Vous ne pouvez pas voir le fond.
- Vous ne pouvez sentir que **la pente sous vos pieds**.

La stratégie :

- 1 Sentez la pente.
- 2 Faites un pas dans la direction qui descend.
- 3 Répétez jusqu'à ce que la pente soit plate.

Gradient

$\nabla_w J$ pointe dans la direction de la *montée* la plus raide. Pour descendre, on va à l'opposé.

Règle de mise à jour

À chaque itération, on avance les paramètres à l'opposé du gradient de la perte, d'un pas ajusté par le **taux d'apprentissage** $\alpha > 0$:

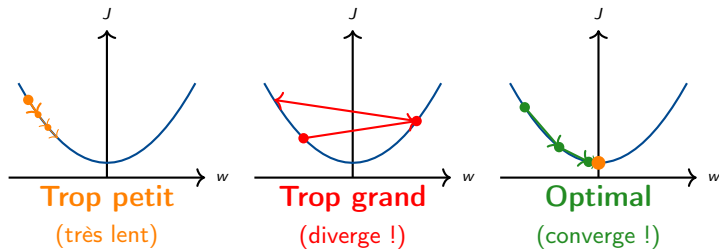
$$w \leftarrow w - \alpha \nabla_w J(w)$$

- J désigne ici **n'importe quelle** perte à minimiser : le critère du perceptron, l'entropie croisée, ou la somme des carrés.
- L'algorithme est toujours le même : initialiser w , puis répéter jusqu'à convergence.
- Seul le calcul de $\nabla_w J$ change d'un modèle à l'autre.

α est critique

Trop grand, les itérés peuvent **diverger**. Trop petit, la convergence se traîne.

Taux d'apprentissage en image



Diagnostic n° 1

Tracer J au fil des itérations. Une courbe qui remonte signale un α trop grand.

Ce que veut dire « jusqu'à convergence »

- On s'arrête quand $\|\nabla J\| < \varepsilon$, ou que le coût ne bouge plus d'une itération à l'autre.
- Un **plafond d'itérations** sert de garde-fou : sans lui, un α mal réglé tourne indéfiniment.
- Pour un problème **convexe** (les moindres carrés, l'entropie croisée), tout minimum est global : on converge vers *la* solution.
- Pour un problème non convexe (les réseaux profonds), on converge vers *un* minimum local, et c'est souvent suffisant.

Par lots ou stochastique

Le gradient est une somme sur les m exemples. Deux façons de la consommer à chaque pas :

Par lots (batch)

- Le gradient sur **tout** l'ensemble d'entraînement avant chaque pas.
- Une descente lisse et directe.
- Relit chaque exemple à chaque fois : impraticable si m est grand.

Stochastique (SGD)

- **Un** exemple à la fois, tiré au hasard.
- Mises à jour ultra-rapides, peu de mémoire.
- Bruitée : oscille autour du minimum sans s'y fixer.

SGD ne se stabilise jamais à pas constant

Le bruit du gradient ne s'annule pas au minimum. On fait donc **décroître** α au fil des itérations (*learning schedule*).

Le mini-batch s'est imposé

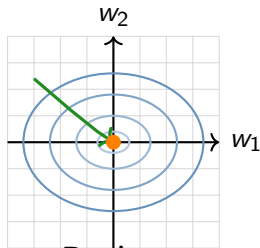
Méthode : prendre un petit lot d'exemples (typiquement 32, 64 ou 128) par pas.

- Plus rapide que le batch, plus stable que le SGD.
- Permet les **opérations matricielles**, donc l'accélération GPU.

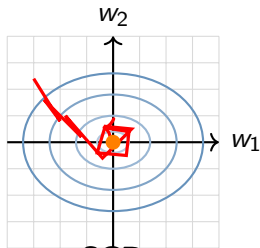
Une époque (epoch)

Un passage complet sur les m exemples. Avec des lots de taille b , cela fait m/b mises à jour par époque : $m = 60\,000$ et $b = 32$ donnent 1875 mises à jour, là où le batch n'en ferait qu'une seule.

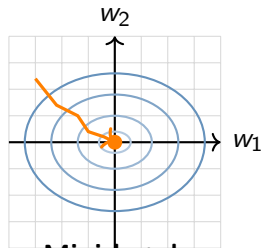
Trajectoires de convergence



Par lots
Lisse et direct



SGD
Oscille sans se fixer



Mini-batch
Équilibré

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron**
- 5 Algo 3 : régression logistique
- 6 Récapitulatif

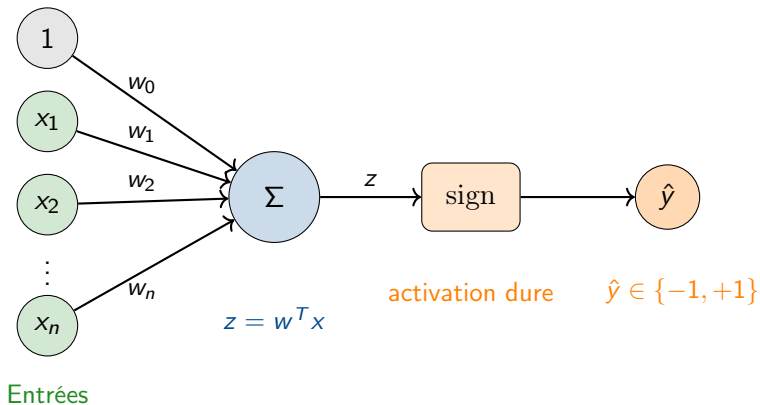
Le modèle tient en un neurone

Le perceptron, sans hypothèse, prend la définition du classifieur linéaire **au pied de la lettre** : un produit scalaire suivi d'une activation dure. C'est le neurone historique.

$$h_w(x) = \text{sign}(w^T x), \quad y \in \{-1, +1\}$$

- Le modèle est exactement celui du séparateur linéaire de la section 1.
- Toute la différence est dans la **façon d'ajuster** w .

Perceptron en image



Les entrées pondérées sont sommées dans le score $z = w^T x$, puis passées dans une activation signe. Ce signe coupe l'espace d'entrée le long de l'hyperplan $w^T x = 0$.

Le critère pénalise chaque point mal classé selon la distance à laquelle il se trouve du **mauvais** côté. Une erreur signifie $y^{(i)} w^T x^{(i)} < 0$, donc sur l'ensemble $\mathcal{M}$ des points mal classés :

$$E(w) = - \sum_{i \in \mathcal{M}} y^{(i)} w^T x^{(i)}$$

- Toujours **positif** : chaque terme de la somme est négatif.
- **Linéaire par morceaux**, donc de gradient bien défini là où il compte.
- Les points **bien** classés ne contribuent pas du tout.

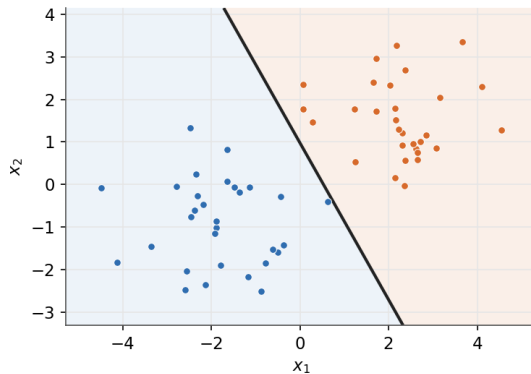
Mise à jour du perceptron

Le critère est une somme sur les points mal classés. En **stochastique**, on n'en prend qu'un terme à la fois : $E^{(i)}(w) = -y^{(i)} w^T x^{(i)}$, dont le gradient vaut $\nabla_w E^{(i)} = -y^{(i)} x^{(i)}$.

Un exemple **bien** classé n'apparaît pas dans la somme : rien à corriger, w ne bouge pas. Sur une **erreur** :

$$w \leftarrow w - \alpha \left(-y^{(i)} x^{(i)} \right) = w + \alpha y^{(i)} x^{(i)}$$

Frontière trouvée



Le perceptron trouve un hyperplan séparateur, pas nécessairement celui de plus grande marge.

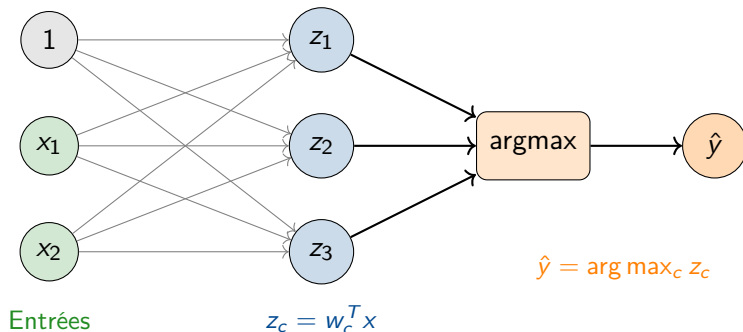
Perceptron multiclasse

Avec k classes, on garde un vecteur de poids w_c **par classe** et on prédit celle au plus fort score. Sur une erreur, on récompense la vraie classe et on pénalise la classe prédite :

$$\hat{y} = \arg \max_c w_c^T x, \quad w_y \leftarrow w_y + \alpha x, \quad w_{\hat{y}} \leftarrow w_{\hat{y}} - \alpha x$$

- Les w_c en colonnes d'une matrice $W \in \mathbb{R}^{(n+1) \times k}$: un seul produit $W^T x$ calcule tous les scores.
- Les scores découpent l'espace d'entrée en k régions, chacune revendiquée par la classe au score le plus élevé.
- Le *signe* du binaire est remplacé par un **argmax**.

Perceptron multiclasse en image



Un neurone de score par classe, et un argmax au sommet. Chaque colonne de W est l'hyperplan (normale et biais) d'une classe.

Exemple

Avec $k = 3$ classes et l'entrée $x = (1,1, -2,0)$, augmentée de $x_0 = 1$:

$$W^T x = \begin{bmatrix} -2 & -4 & 1 \\ -4 & 2 & 4 \\ -6 & 4 & -5 \end{bmatrix} \begin{bmatrix} 1 \\ 1,1 \\ -2,0 \end{bmatrix} = \begin{bmatrix} -8,4 \\ -9,8 \\ 8,4 \end{bmatrix}$$

- Le troisième score l'emporte : l'entrée est affectée à la **classe 3**.
- En lisant la troisième ligne, $w_3^T x = -6 + 4 \times 1,1 + (-5) \times (-2,0) = 8,4$.
- Chaque **ligne** de W^T est l'hyperplan d'une classe : son premier coefficient est le biais, les suivants la normale.

- **Séparables** : nombre fini de mises à jour, arrêt sur un hyperplan séparateur (*théorème de convergence du perceptron*).
- **Non séparables** : jamais d'arrêt, et **rien ne signale** ce cas.

Tous les séparateurs se valent

$E(w) = 0$ dès qu'il ne reste plus d'erreur, même pour un hyperplan qui frôle les données.

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique**
- 6 Récapitulatif

Une activation lisse

Garder le neurone, remplacer l'échelon dur par la **sigmoïde** lisse. La sortie devient la probabilité de la classe positive, avec $y \in \{0, 1\}$:

$$\hat{y} = p(y = 1 \mid x; w) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

- Le $\hat{y}$ du perceptron était une **classe dure**. Celui-ci est une **probabilité**.
- La seuiller à $\frac{1}{2}$ redonne une classe, et la **même** frontière $w^T x = 0$, puisque $\sigma(z) \geq \frac{1}{2} \iff z \geq 0$.
- Le modèle n'a pas changé de forme. Seule l'activation s'est adoucie.

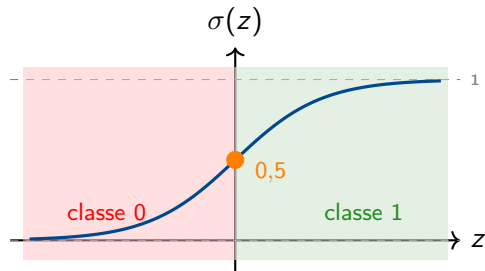
Sigmoïde

Une fonction qui envoie $(-\infty, +\infty)$ dans $(0, 1)$:

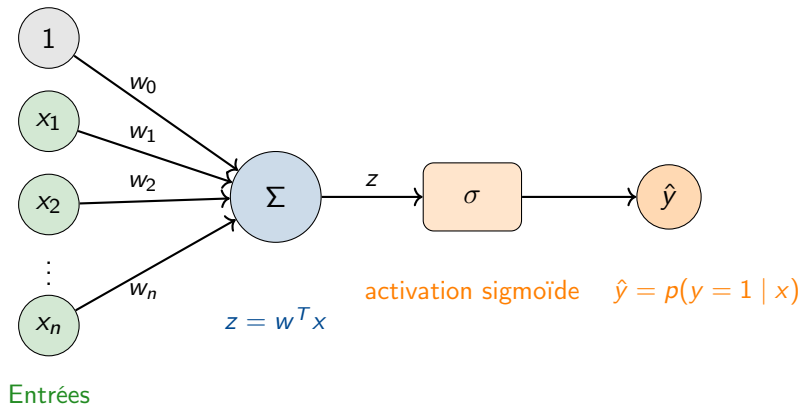
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z = 0 \Rightarrow \sigma = 0,5$
- $z \rightarrow +\infty \Rightarrow \sigma \rightarrow 1$
- $z \rightarrow -\infty \Rightarrow \sigma \rightarrow 0$

Le seuil $\sigma = 0,5$ correspond exactement à $w^T x = 0$.



Neurone adouci



Le même neurone, l'échelon remplacé par la sigmoïde : la sortie devient une probabilité, et la seuiller à $\frac{1}{2}$ redonne la même frontière $w^T x = 0$.

Pourquoi la sigmoïde ?

Ce n'est pas un choix de compression arbitraire. Écrire l'a posteriori avec la **règle de Bayes** donne exactement cette forme :

$$p(C_1 | x) = \frac{1}{1 + e^{-a}}, \quad a = \ln \frac{p(x | C_1) p(C_1)}{p(x | C_0) p(C_0)}$$

- a est le **log-rapport de vraisemblance**, a priori compris : exactement la quantité que la formulation probabiliste compare entre deux classes.
- La sigmoïde est donc la fonction qui transforme ce rapport en probabilité.

Conséquence

Une sortie logistique bien entraînée **est** une probabilité a posteriori, pas un score qu'on interprète librement comme tel.

Chaque étiquette est vue comme un tirage de **Bernoulli**. Sa vraisemblance, passée au $-\log$, donne la perte à minimiser $L(w)$ (le J de la descente de gradient) :

$$L(w) = - \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

La perte n'est pas choisie, elle est déduite

L'hypothèse sur les étiquettes donne la perte : Bernoulli donne l'entropie croisée, comme le bruit gaussien donnait les moindres carrés.

Preuve : le maximum de vraisemblance donne l'entropie croisée

Mise en place

Chaque étiquette est un tirage de Bernoulli de probabilité de succès $\hat{y}^{(i)} = \sigma(w^T x^{(i)})$. Les deux cas se replient en une seule expression, car $y^{(i)} \in \{0, 1\}$ sélectionne le facteur :

$$p(y^{(i)} | x^{(i)}; w) = (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}$$

Les exemples sont i.i.d., la vraisemblance se factorise, et le logarithme fait descendre les exposants :

$$\ell(w) = \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

Maximiser ℓ revient à minimiser $-\ell$, qui est exactement $L(w)$.

Aucune forme close mais un gradient limpide

Contrairement aux moindres carrés, cette perte n'a **pas** de minimiseur en forme close : la sigmoïde rend les équations de stationnarité transcendantes. On revient donc à la descente de gradient.

Mais dériver récompense l'effort : presque tout se simplifie et le gradient se réduit au **résidu** fois l'entrée :

$$w_j \leftarrow w_j - \alpha \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

C'est exactement la forme de la mise à jour du perceptron dans son codage $\{0, 1\}$. Seule l'activation qui produit $\hat{y}$ a changé.

Preuve (1/2) : la sigmoïde se dérive en elle-même

Identité clé

Notons $z^{(i)} = w^T x^{(i)}$, de sorte que $\hat{y}^{(i)} = \sigma(z^{(i)})$. Toute la dérivation repose sur une seule identité :

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) (1 - \sigma(z))$$

Sur la perte d'un seul exemple, la règle de dérivation en chaîne a trois étages, de la perte à la sortie, de la sortie au score, du score au poids :

$$\frac{\partial L^{(i)}}{\partial \hat{y}^{(i)}} = -\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)} (1 - \hat{y}^{(i)})}, \quad \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \hat{y}^{(i)} (1 - \hat{y}^{(i)}), \quad \frac{\partial z^{(i)}}{\partial w_j} = x_j^{(i)}$$

Preuve (2/2) : tout se simplifie

Le dénominateur du premier facteur est **exactement** le deuxième facteur. En multipliant les trois :

$$\frac{\partial L^{(i)}}{\partial w_j} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)} (1 - \hat{y}^{(i)})} \cdot \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \cdot x_j^{(i)} = (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

Conclusion

Sommer sur l'ensemble d'entraînement donne $\partial L / \partial w_j = \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$, et brancher ce gradient dans $w \leftarrow w - \alpha \nabla_w L$ est la mise à jour encadrée. ■

Le facteur $\hat{y}(1 - \hat{y})$, qui aurait écrasé le gradient là où la sigmoïde sature, disparaît précisément parce que la perte est l'entropie croisée et non l'erreur quadratique.

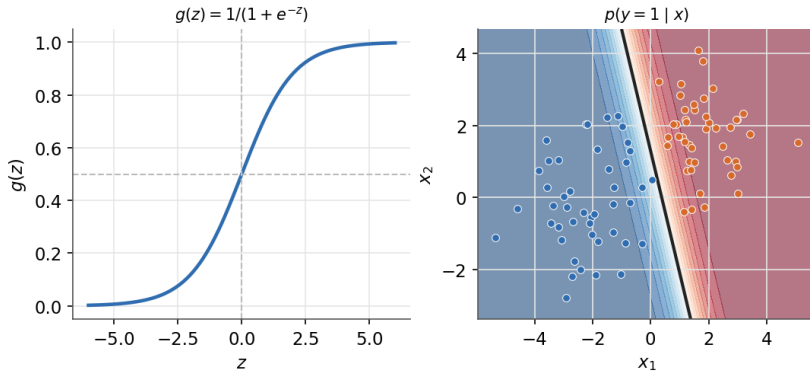
Chaque point tire selon son résidu

Différence avec le perceptron

Le gradient logistique fait intervenir **chaque** point d'entraînement, pas seulement les mal classés. Chaque point tire proportionnellement à son résidu $\hat{y}^{(i)} - y^{(i)}$.

- Un point bien classé et confiant a un résidu presque nul : il ne perturbe presque rien, contrairement aux moindres carrés, où il tirait fort.
- Un point mal classé tire fort, mais de façon **continue**, pas par sauts.
- C'est ce qui rend la régression logistique **stable**, et utilisable sur des données non séparables, là où le perceptron oscille sans fin.

Sigmoïde et frontière de décision



À gauche, la sigmoïde envoie tout score dans l'intervalle $(0, 1)$. À droite, la frontière de décision et la probabilité prédite.

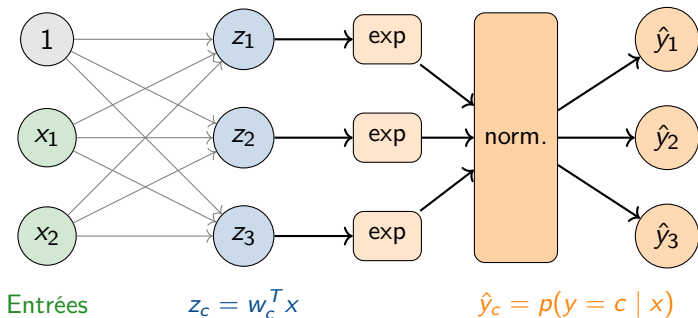
Softmax en multiclasse

Pour k classes, la sigmoïde se généralise en la **softmax** : un score par classe, des exponentielles qui rendent les scores positifs, une normalisation qui en fait une distribution.

$$\hat{y}_c = p(y = c \mid x; w) = \frac{\exp(w_c^T x)}{\sum_{j=1}^k \exp(w_j^T x)}$$

- Les $\hat{y}_c$ sont positifs et somment à 1 : c'est bien une distribution sur les classes.
- Avec des étiquettes one-hot, la perte est l'entropie croisée catégorielle
 $L = -\sum_i \log p(y^{(i)} \mid x^{(i)})$.
- Son gradient garde la **même forme** : le résidu fois l'entrée.

Softmax en réseau



Les exponentielles rendent chaque score positif, la normalisation les fait sommer à 1. C'est le réseau du perceptron multiclasse, l'argmax remplacé par une tête lisse et dérivable.

	sigmoïde	softmax
classes	2	k
sortie	une probabilité $\hat{y}$	une distribution sur k classes
relation	la softmax à $k = 2$ s'y réduit	généralise la sigmoïde

- Décaler chaque w_c du **même** vecteur laisse la softmax inchangée : seule la *différence* des scores compte.
- C'est pourquoi le cas binaire n'a besoin que d'un seul vecteur de poids.

Preuve : la softmax à $k = 2$ est la sigmoïde

Deux classes pour deux scores

La softmax note l'entrée deux fois, w_1 pour la classe positive et w_0 pour la négative.

$$p(y = 1 \mid x; w) = \frac{e^{w_1^T x}}{e^{w_1^T x} + e^{w_0^T x}}$$

Diviser le numérateur et le dénominateur par $e^{w_1^T x}$ laisse

$$p(y = 1 \mid x; w) = \frac{1}{1 + e^{-(w_1 - w_0)^T x}} = \sigma((w_1 - w_0)^T x)$$

Seule la différence $w = w_1 - w_0$ compte : un seul vecteur de poids suffit, exactement le modèle binaire du début de section.

Plan du cours

- 1 Séparateur linéaire
- 2 Algo 1 : moindres carrés comme classifieur
- 3 Descente de gradient
- 4 Algo 2 : perceptron
- 5 Algo 3 : régression logistique
- 6 Récapitulatif**

Formule

Hypothèse sur les données	classes de forme gaussienne
Activation	identité pour entraîner (cibles ± 1), $\text{sign}(w^T x)$ pour prédire
Perte	erreur quadratique $\sum_{i=1}^m (w^T x^{(i)} - y^{(i)})^2$
Ajustement	forme close $w = (X^T X)^{-1} X^T y$, aucune itération
Multiclasse	lignes one-hot de Y , $W = (X^T X)^{-1} X^T Y$, prédire $\arg \max_k (W^T x)_k$

La méthode avec hypothèse et forme close.

	Formule
Hypothèse sur les données	aucune
Activation	échelon, $h_w(x) = \text{sign}(w^T x)$, $y \in \{-1, +1\}$
Perte	$E(w) = -\sum_{i \in \mathcal{M}} y^{(i)} w^T x^{(i)}$, points mal classés seulement
Gradient	$\nabla_w E = -\sum_{i \in \mathcal{M}} y^{(i)} x^{(i)}$
Mise à jour	$w \leftarrow w + \alpha y^{(i)} x^{(i)}$ sur une erreur, rien sinon
Multiclasse	$\hat{y} = \arg \max_c w_c^T x$, puis $w_y += \alpha x$ et $w_{\hat{y}} -= \alpha x$
Convergence	finie si les données sont séparables, oscille sinon

Guidé par les erreurs, et sans hypothèse.

	Formule
Hypothèse sur les données	aucune
Activation	sigmoïde, $\hat{y} = \sigma(w^T x)$, une probabilité, $y \in \{0, 1\}$
Perte	entropie croisée $-\sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$
Gradient	$\nabla_{w_j} L = \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$, chaque point tire
Mise à jour	$w_j \leftarrow w_j - \alpha \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$
Multiclasse	softmax $\hat{y}_c = \frac{\exp(w_c^T x)}{\sum_j \exp(w_j^T x)}$, entropie croisée catégorielle

Probabiliste, et sans hypothèse.

Trois activations pour une même règle

Lues côte à côte, la mise à jour du perceptron (dans son codage $\{0, 1\}$) et celle de la régression logistique sont **la même formule**, le résidu fois l'entrée :

$$w_j \leftarrow w_j + \alpha \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

$\hat{y}$ produit par	Modèle	Cible
l' échelon $\text{sign}(w^T x)$	le perceptron	une classe
la sigmoïde $\sigma(w^T x)$	la régression logistique	une probabilité
l' identité $w^T x$	la régression linéaire	un nombre

Seule l'activation change

La règle d'apprentissage, elle, ne bouge pas. Le deep learning empile précisément ces unités en couches.

- Un classifieur linéaire lit le **signe** du score $w^T x$, et la frontière est l'hyperplan $w^T x = 0$.
- Les **moindres carrés** classifient en forme close, mais paient une hypothèse gaussienne : les points les plus sûrs tirent la frontière.
- Le **perceptron** ne corrige que ses erreurs, converge si les données sont séparables, et ne choisit pas le meilleur hyperplan parmi ceux qui séparent.
- La **régression logistique** sort une probabilité, et son gradient est le **résidu fois l'entrée** : chaque point compte, proportionnellement à son erreur.
- La **descente de gradient** remplace la forme close partout où elle n'existe pas.